TP 3 : un même traitement, quatre moteurs

En binôme. Plateforme : **Google Cloud** (crédits Éducation ou essai) : une VM Compute Engine, puis un cluster **Managed Service for Apache Spark** (ex-Dataproc).
Prérequis : cours de la séance 3 ; TP 2 (table `gold_payment_features`) ; un projet GCP avec facturation ; `gcloud` disponible (Cloud Shell suffit).

> Contexte : au TP 2, vous avez calculé les features de fraude de ShopFlow sur Databricks, à l'échelle SF1. ShopFlow grandit : à sf10, il y a 10 millions de paiements et 160 millions d'événements de navigation. L'équipe fraude veut recalculer chaque nuit les features de tout l'historique, en y ajoutant un signal tiré du clickstream : **le nombre de passages en caisse (`checkout`) depuis la même adresse IP juste avant le paiement**, qui trahit le test de cartes volées. Vous allez écrire ce calcul avec quatre outils, mesurer temps, mémoire et coût, et décider lequel ShopFlow devrait utiliser.

Le calcul de référence

Entrées : `payments` (colonnes utiles : `payment_id`, `card_id`, `payment_ts`, `amount`, `ip`, et `fraud_scenario` pour le seul point de contrôle) et `clickstream` (`ip`, `event_ts`, `event_type`).

Le calcul de référence (suite)

1. **Table `ip_hourly`**, une ligne par couple (`ip`, `hour`), où `hour` est `event_ts` tronqué à l'heure : `n_events` (nombre d'événements), `n_checkouts` (nombre d'événements `checkout`).
2. **Features par carte**, une ligne par paiement, mêmes définitions qu'au TP 2. La clé de carte est `card_key = coalesce(card_id, -payment_id)` (un paiement PayPal, sans carte, forme sa propre clé). Les fenêtres portent sur le passé de la carte, paiement courant inclus, bornes incluses :
   - `card_nb_tx_1d`, `card_nb_tx_7d` : nombre de paiements de la carte sur 1 et 7 jours ;
   - `card_avg_amount_30d` : montant moyen de la carte sur 30 jours ;
   - `amount_ratio_30d` = `amount / card_avg_amount_30d`.
3. **Table `payment_features`** : les features par carte, plus `ip_nb_events_2h` et `ip_nb_checkouts_2h`, sommes de `ip_hourly` pour l'IP du paiement sur **l'heure du paiement et l'heure précédente** (0 si l'IP n'apparaît pas).

Le calcul de référence (suite)

Astuce pour l'étape 3 : dupliquez `ip_hourly` en décalant `hour` d'une heure, additionnez par (`ip`, `hour`), puis faites **une seule jointure à gauche** sur (`ip`, heure du paiement).

Points de contrôle (sf10) : `payment_features` a exactement autant de lignes que `payments` (environ 10,3 millions). Pour la plupart des paiements, `ip_nb_checkouts_2h` vaut 1 (le passage en caisse du paiement lui-même) ; pour le scénario `test_de_cartes`, il doit être nettement plus élevé.

Le calcul de référence (suite)

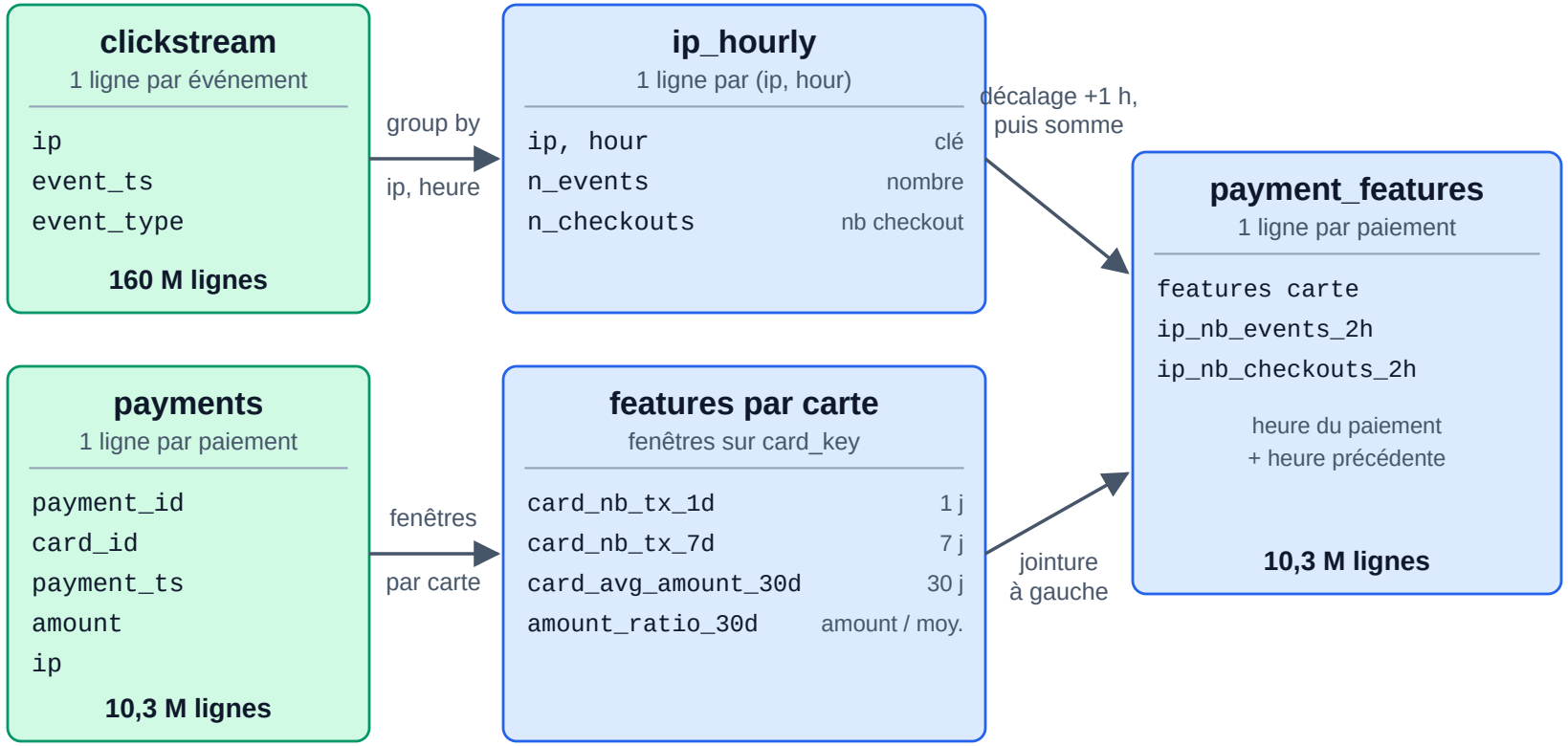


Schéma : deux calculs indépendants, une agrégation du clickstream par IP et par heure et des fenêtres glissantes par carte, réunis par une jointure ; les volumes sont ceux de sf10.

Le calcul de référence (suite)

Livrable : vos scripts et un fichier `mesures.md` avec la grille de la partie 4 et vos réponses aux questions ❓.

> 💸 **Coût** : une VM ou un cluster allumé consomme vos crédits même inutilisé. **Supprimez tout en fin de séance** (dernière étape). Les quotas de vCPU sont limités : on supprime la VM **avant** de créer le cluster.

Le calcul de référence (suite)

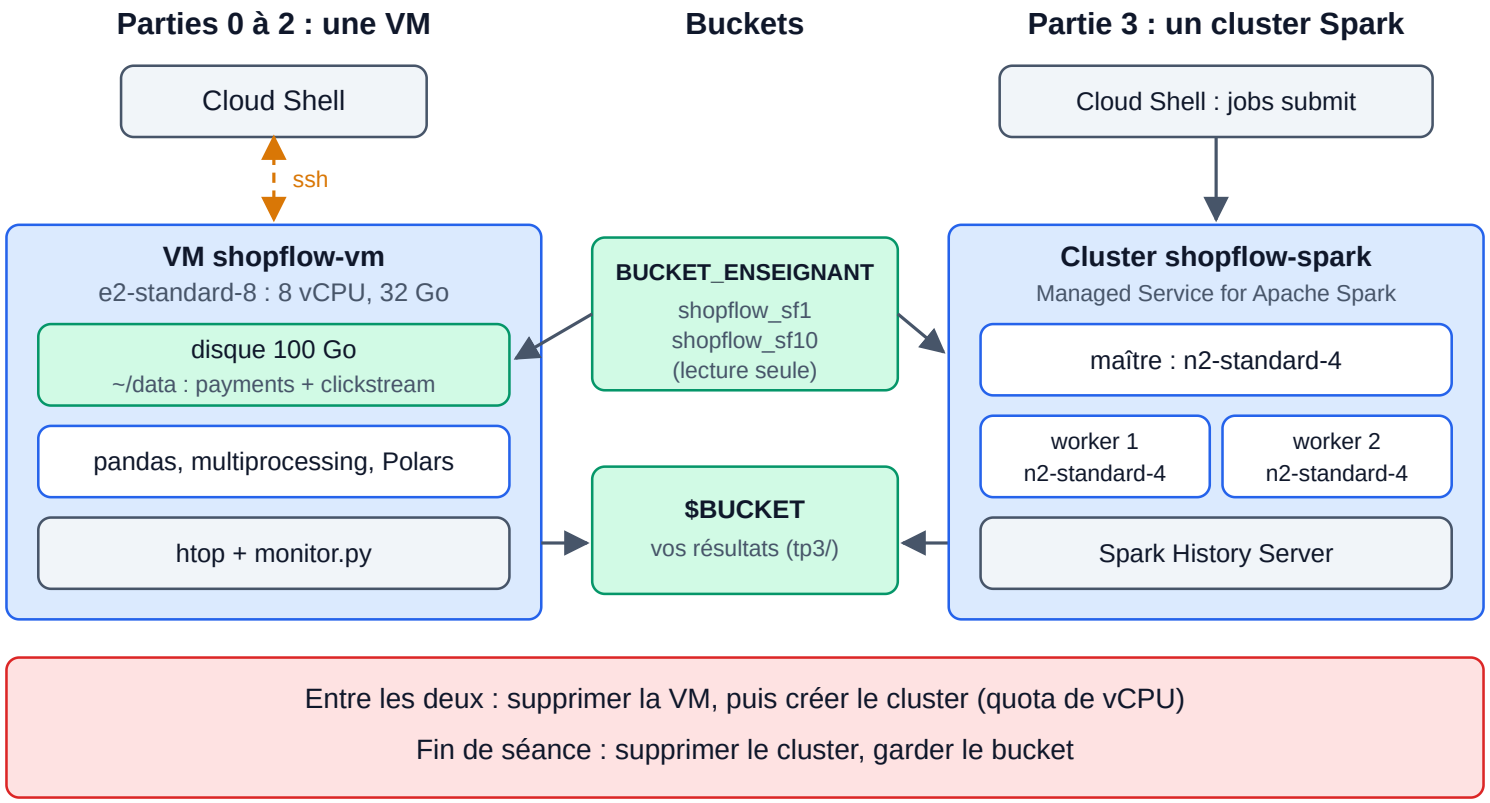


Schéma : la VM copie les données sur son disque, le cluster les lit directement dans le bucket de l'enseignant ; vos résultats vont toujours dans votre bucket.

Partie 0 : la VM et les données

1. Dans Cloud Shell, réglez la région et créez un bucket pour vos résultats (le même nom servira aux TP 4 et 5) :

```
gcloud config set compute/region europe-west1
gcloud config set compute/zone europe-west1-b
export BUCKET=gs://$(gcloud config get-value project)-shopflow
gcloud storage buckets create $BUCKET --location=europe-west1
```

   Activez aussi les API utilisées dans le TP et donnez deux rôles au compte de service par défaut des VM : les machines du cluster Spark (partie 3) s'exécutent sous ce compte. `roles/dataproc.worker` lui permet de faire tourner le cluster, `roles/storage.objectAdmin` de lire et d'écrire dans les buckets que Dataproc crée pour lui (`dataproc-staging-…`, `dataproc-temp-…`) et dans le vôtre. Sur les projets récents, ce compte n'a plus automatiquement le rôle Éditeur, d'où cette étape :

Partie 0 : la VM et les données (suite)

```
gcloud services enable compute.googleapis.com dataproc.googleapis.com cloudresourcemanager.googleapis.com
PROJECT=$(gcloud config get-value project)
PN=$(gcloud projects describe $PROJECT --format='value(projectNumber)')
SA=serviceAccount:$PN-compute@developer.gserviceaccount.com
for ROLE in roles/dataproc.worker roles/storage.objectAdmin; do
  gcloud projects add-iam-policy-binding $PROJECT --member=$SA --role=$ROLE --condition=None
done
```

Les droits mettent une à deux minutes à s'appliquer : c'est fait bien avant la partie 3.

2. Créez la VM (8 vCPU, 32 Go de RAM) :

```
gcloud compute instances create shopflow-vm \
  --machine-type=e2-standard-8 \
  --image-family=debian-12 --image-project=debian-cloud \
  --boot-disk-size=100GB --scopes=cloud-platform
```

Partie 0 : la VM et les données (suite)

3. Connectez-vous à la VM :

```
gcloud compute ssh shopflow-vm
```

4. Sur la VM, installez l'environnement et copiez les données SF1 et sf10 (environ 4 Go au total) depuis le bucket de l'enseignant :

Partie 0 : la VM et les données (suite)

```
sudo apt-get update && sudo apt-get install -y python3-venv htop
python3 -m venv ~/venv && source ~/venv/bin/activate
echo 'source ~/venv/bin/activate' >> ~/.bashrc   # réactive le venv à chaque connexion SSH
python -m pip install -U pip
python -m pip install pandas pyarrow polars psutil
which python      # doit afficher /home/<votre_login>/venv/bin/python
python -c "import pandas, pyarrow, polars, psutil; print('environnement OK')"
for sf in 1 10; do
  mkdir -p ~/data/sf$sf
  gcloud storage cp -r gs://shopflow-esigelec-big-data-ia/shopflow_sf$sf/payments gs://shopflow-esigelec-big-data-ia/shopflow_sf$sf/clickstream ~/data/sf$sf/
done
ls -lh ~/data/sf10/payments | head
mkdir -p ~/tp3 && cd ~/tp3        # vos scripts ici, les données restent dans ~/data
```

Partie 0 : la VM et les données (suite)

⚠️ Lancez toujours vos scripts avec `python` **une fois le venv activé** (l'invite affiche `(venv)`). Si vous voyez `ImportError: Unable to find a usable engine; tried using: 'pyarrow', 'fastparquet'` ou un chemin `/usr/lib/python3/...` dans l'erreur, c'est le Python du système qui tourne, pas celui du venv : faites `source ~/venv/bin/activate` et relancez. `pyarrow` est indispensable : c'est lui que pandas utilise pour lire le Parquet.

5. Récupérez le script de mesure `monitor.py` fourni par l'enseignant (ou copiez-le depuis l'annexe). Il enregistre chaque demi-seconde la mémoire utilisée et le CPU de la machine :

```
python monitor.py mesures_pandas.csv &      # démarre la mesure en arrière-plan
python mon_script.py ...                    # votre traitement
kill %1                                     # arrête la mesure
python monitor.py --resume mesures_pandas.csv   # affiche pic RAM, CPU moyen, durée
```

Partie 0 : la VM et les données (suite)

❓ Q0. Combien de fichiers Parquet contiennent `payments` et `clickstream` ? Chacun correspond à un mois : un calcul par IP et par heure peut-il se faire fichier par fichier ? Et une fenêtre de 30 jours par carte ?

Sources : Partie 0 : la VM et les données

Source : créer une VM.

Partie 1 : pandas

Les deux versions pandas (naïve et `multiprocessing`) partagent les mêmes fonctions. Créez `features_lib.py` dans `~/tp3` :

Partie 1 : pandas (suite)

```python
import pandas as pd

PAY_COLS = ["payment_id", "card_id", "payment_ts", "amount", "ip", "fraud_scenario"]
CLK_COLS = ["ip", "event_ts", "event_type"]

def ip_hourly_of(clk: pd.DataFrame) -> pd.DataFrame:
    """Une ligne par (ip, hour) : n_events, n_checkouts."""
    # TODO : hour = event_ts tronqué à l'heure (dt.floor), puis groupby(["ip", "hour"])
    ...

def card_windows(pay: pd.DataFrame) -> pd.DataFrame:
    """Ajoute card_key, card_nb_tx_1d, card_nb_tx_7d, card_avg_amount_30d, amount_ratio_30d."""
    # TODO : trier par (card_key, payment_ts), puis
    #        pay.groupby("card_key").rolling("7D", on="payment_ts", closed="both")["amount"].count()
    #        Attention : l'index du résultat n'est pas celui de pay (il dépend de la version
    #        de pandas). Comment remettre les valeurs dans pay sans dépendre de cet index ?
    ...

def add_ip_features(pay: pd.DataFrame, ip_hourly: pd.DataFrame) -> pd.DataFrame:
    """Ajoute ip_nb_events_2h et ip_nb_checkouts_2h (heure du paiement + heure précédente)."""
    # TODO : l'astuce du décalage d'une heure, puis merge(how="left") et fillna(0)
    ...
```

1.A La version naïve

Écrivez `features_pandas.py` qui lit **tout** `payments` et **tout** `clickstream` d'un coup avec `pd.read_parquet(dossier, columns=...)`, enchaîne les trois fonctions et écrit `payment_features.parquet`.

Lancez-la d'abord sur **SF1** (`~/data/sf1`), avec `monitor.py`. Vérifiez le point de contrôle :

```
import pandas as pd
f = pd.read_parquet("payment_features.parquet")
print(len(f)); print(f.groupby("fraud_scenario", dropna=False).ip_nb_checkouts_2h.mean())
```

Puis lancez-la sur **sf10**, en gardant `htop` ouvert dans un second terminal.

1.A La version naïve (suite)

❓ Q1. Que se passe-t-il sur sf10 ? Laquelle des deux tables d'entrée pèse le plus en mémoire ? Sur SF1, regardez le type de la colonne `ip` (`clk.dtypes`) et son poids (`clk.memory_usage(deep=True)`). Estimez la mémoire nécessaire à sf10 à partir du pic mesuré sur SF1.

1.B La version `multiprocessing`

On veut traiter un mois par processus. Deux difficultés, celles de Q0 :

- les fenêtres de 30 jours d'un paiement du début du mois remontent dans les mois précédents : chaque processus lit donc aussi les paiements des **deux mois précédents** (février est court), calcule les fenêtres, puis ne garde que les paiements de son mois ;
- un paiement de la première heure du mois a besoin de la dernière heure du mois précédent dans le clickstream.

Ajoutez à `features_lib.py` :

1.B La version `multiprocessing` (suite)

```
import os

def features_of_month(month: str, src: str, out: str) -> str:
    """month = '2025-03'. Calcule payment_features pour les paiements du fichier de ce mois,
    écrit le résultat dans out/payment_features/ et renvoie son chemin."""
    start = pd.Timestamp(f"{month}-01", tz="UTC")
    # TODO 1 : lire payments du mois et des deux mois précédents (s'ils existent),
    #          ne garder des mois précédents que payment_ts >= start - 30 jours,
    #          marquer les lignes du mois traité, appliquer card_windows, ne garder qu'elles
    # TODO 2 : lire clickstream du mois, et du mois précédent seulement event_ts >= start - 1 h
    #          (pd.read_parquet(..., filters=[("event_ts", ">=", start - pd.Timedelta(hours=1))]))
    # TODO 3 : add_ip_features, écrire le Parquet, renvoyer le chemin
    ...
```

Et le script `features_multiproc.py` :

1.B La version `multiprocessing` (suite)

```
import glob, os, sys
from concurrent.futures import ProcessPoolExecutor
from functools import partial
import features_lib

if __name__ == "__main__":
    src, out, workers = sys.argv[1], sys.argv[2], int(sys.argv[3])
    os.makedirs(f"{out}/payment_features", exist_ok=True)
    months = sorted(os.path.basename(f)[5:12] for f in glob.glob(f"{src}/payments/part-*.parquet"))
    with ProcessPoolExecutor(max_workers=workers) as pool:
        parts = list(pool.map(partial(features_lib.features_of_month, src=src, out=out), months))
    print(len(parts), "mois traités")
```

1.B La version `multiprocessing` (suite)

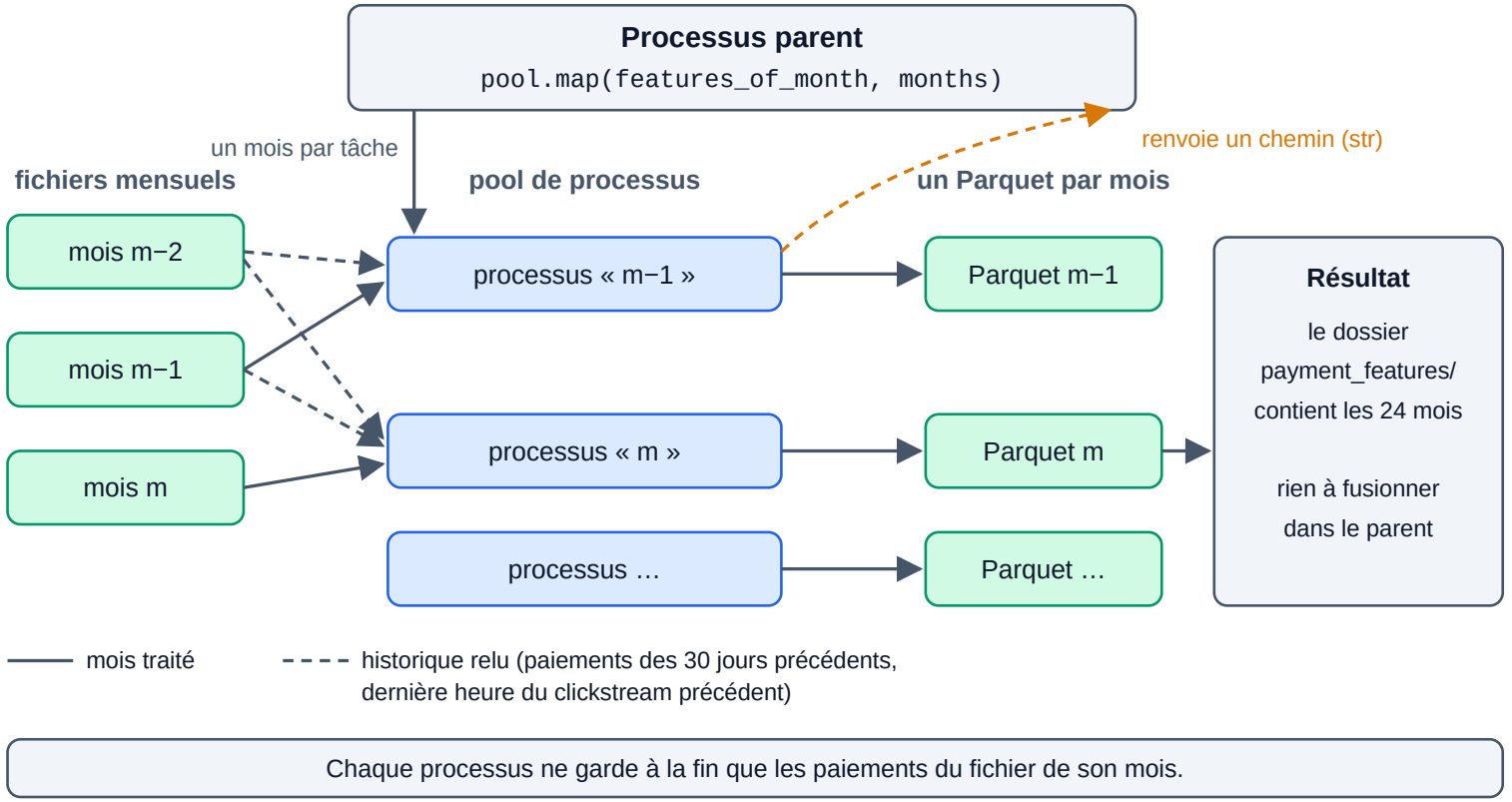


Schéma : ce que fait le squelette ci-dessus ; chaque processus relit un peu du passé pour ses fenêtres, et seul un chemin revient au parent.

1.B La version `multiprocessing` (suite)

Vérifiez sur SF1 que le résultat est identique à la version naïve (même nombre de lignes, mêmes moyennes par scénario, à quelques paiements près autour de minuit en fin de mois), puis mesurez sur sf10 avec 2, 4 puis 8 processus.

❓ Q2. Comment évoluent la durée et le pic de RAM avec le nombre de processus ? Pourquoi la fonction renvoie-t-elle un chemin plutôt qu'un DataFrame ? Quelle part des données chaque processus lit-il « en trop » à cause de l'historique ?

Sources : 1.B La version `multiprocessing`

Sources : `concurrent.futures` ; pandas : fenêtres temporelles et `groupby().rolling()` ; `read_parquet` et ses filtres.

Partie 2 : Polars

Écrivez `features_polars.py` avec l'API *lazy* :

- `pl.scan_parquet(f"{src}/payments/*.parquet")` et `pl.scan_parquet(f"{src}/clickstream/*.parquet")` ;
- `ip_hourly` : `group_by("ip", pl.col("event_ts").dt.truncate("1h").alias("hour")).agg(...)` ;
- fenêtres par carte : expressions `rolling_sum_by` et `rolling_mean_by` sur `payment_ts` (`closed="both"`), évaluées par carte avec `.over("card_key")` ;
- l'astuce du décalage : `pl.concat([ip_hourly, ip_hourly.with_columns(pl.col("hour") + pl.duration(hours=1))])` ;
- `collect(engine="streaming")` à la fin.

1. Affichez le plan optimisé avec `.explain()` **avant** d'exécuter. Repérez les colonnes réellement lues dans chaque fichier.
2. Vérifiez sur SF1 le point de contrôle, puis mesurez sur sf10.

Partie 2 : Polars (suite)

❓ Q3. Combien de colonnes Polars lit-il dans `clickstream` ? Comparez durée et pic de RAM avec `multiprocessing`. Combien de cœurs Polars utilise-t-il (regardez `htop`) ? Polars a-t-il eu besoin de vos précautions d'historique de la partie 1 ? Pourquoi ?

Sources : Partie 2 : Polars

Sources : Polars lazy API ; streaming ; `rolling_sum_by` ; fenêtres `over`.

Pause et changement d'infrastructure

1. Copiez vos scripts et mesures dans votre bucket : `gcloud storage cp *.csv *.py $BUCKET/tp3/`.
2. **Supprimez la VM**, puis vérifiez qu'elle n'apparaît plus : ses 8 vCPU comptent dans votre quota tant qu'elle existe.

```
gcloud compute instances delete shopflow-vm --quiet
gcloud compute instances list          # doit être vide
```

3. Regardez combien de vCPU votre projet peut allumer **au total, toutes régions confondues** (quota `CPUS_ALL_REGIONS`). Un compte d'essai est limité à 8 ou 12 vCPU :

```
gcloud compute project-info describe --flatten=quotas \
  --filter="quotas.metric=CPUS_ALL_REGIONS" --format="table(quotas.limit,quotas.usage)"
```

Pause et changement d'infrastructure (suite)

4. Créez le cluster Spark : un nœud maître et deux workers (Dataproc impose au moins 2 workers). Choisissez la taille des machines selon la limite affichée :
   - limite de **12 vCPU ou plus** : `TYPE=n2-standard-4` (3 × 4 = 12 vCPU, 16 Go de RAM par nœud) ;
   - limite de **8 vCPU** : `TYPE=e2-standard-2` (3 × 2 = 6 vCPU, 8 Go de RAM par nœud ; le job sera plus lent, notez-le dans la grille).

```
TYPE=n2-standard-4        # ou e2-standard-2, voir ci-dessus
gcloud dataproc clusters create shopflow-spark \
  --region=europe-west1 \
  --image-version=3.0-debian13 \
  --master-machine-type=$TYPE --master-boot-disk-size=50GB \
  --num-workers=2 --worker-machine-type=$TYPE --worker-boot-disk-size=50GB \
  --enable-component-gateway \
  --delete-max-idle=30m
```

Pause et changement d'infrastructure (suite)

Si l'erreur `Insufficient 'CPUS_ALL_REGIONS' quota. Requested X, available Y` apparaît, une autre machine tourne encore (refaites `gcloud compute instances list`) ou les machines sont trop grosses : il faut que 3 × (vCPU d'une machine) ≤ Y.

`--delete-max-idle` supprime le cluster après 30 min d'inactivité, une sécurité contre l'oubli.

Sources : créer un cluster ; comptes de service Dataproc ; quotas Compute Engine ; référence `gcloud dataproc clusters create` ; suppression planifiée ; versions d'image.

Partie 3 : PySpark sur cluster

3.A Le calcul

La VM n'existe plus : travaillez désormais dans **Cloud Shell**. Créez le script dans un dossier `~/tp3` de Cloud Shell (`mkdir -p ~/tp3 && cd ~/tp3`, puis `cloudshell edit features_spark.py` ou `nano features_spark.py`). Inutile de le copier sur le cluster : à la soumission, `gcloud` envoie le fichier local dans le bucket de staging de Dataproc et le lance sur le cluster. Si Cloud Shell a redémarré, redéfinissez `BUCKET` (étape 1 de la partie 0).

Le script `features_spark.py` est fourni : copiez-le tel quel. Il lit `payments` et `clickstream` dans le bucket de l'enseignant (premier argument), calcule `payment_features` avec l'API DataFrame (le même calcul de référence qu'en pandas et Polars) et l'écrit dans votre bucket (second argument). Prenez le temps de le lire et de relier chaque bloc à une étape du calcul de référence : la question Q4 s'appuie dessus.

3.A Le calcul (suite)

```
import sys
from pyspark.sql import SparkSession, Window, functions as F

spark = SparkSession.builder.appName("shopflow-features").getOrCreate()
src, out = sys.argv[1], sys.argv[2]

# Étape 2 : features par carte (fenêtres glissantes sur le temps, comme au TP 2)
pay = (spark.read.parquet(f"{src}/payments")
       .select("payment_id", "card_id", "payment_ts", "amount", "ip", "fraud_scenario")
       .withColumn("card_key", F.coalesce("card_id", -F.col("payment_id")))
       .withColumn("t", F.col("payment_ts").cast("long")))       # secondes depuis 1970

def w(days: int):
    """Fenêtre : même carte, du paiement courant jusqu'à `days` jours avant."""
    return Window.partitionBy("card_key").orderBy("t").rangeBetween(-days * 86400, 0)

pay = (pay.withColumn("card_nb_tx_1d", F.count(F.lit(1)).over(w(1)))
          .withColumn("card_nb_tx_7d", F.count(F.lit(1)).over(w(7)))
          .withColumn("card_avg_amount_30d", F.avg("amount").over(w(30)))
          .withColumn("amount_ratio_30d", F.col("amount") / F.col("card_avg_amount_30d")))

# Étape 1 : ip_hourly, une ligne par (ip, heure)
```

3.A Le calcul (suite)

```
clk = spark.read.parquet(f"{src}/clickstream").select("ip", "event_ts", "event_type")
ip_hourly = (clk.groupBy("ip", F.date_trunc("hour", "event_ts").alias("hour"))
                .agg(F.count(F.lit(1)).alias("n_events"),
                     F.sum((F.col("event_type") == "checkout").cast("int")).alias("n_checkouts")))

# Astuce de l'étape 3 : chaque heure compte aussi pour l'heure suivante
two_h = (ip_hourly.unionByName(ip_hourly.withColumn("hour", F.col("hour") + F.expr("INTERVAL 1 HOUR")))
           .groupBy("ip", "hour")
           .agg(F.sum("n_events").alias("ip_nb_events_2h"),
                F.sum("n_checkouts").alias("ip_nb_checkouts_2h")))

# Étape 3 : une seule jointure à gauche sur (ip, heure du paiement)
res = (pay.withColumn("hour", F.date_trunc("hour", "payment_ts"))
          .join(two_h, ["ip", "hour"], "left")
          .fillna(0, subset=["ip_nb_events_2h", "ip_nb_checkouts_2h"])
          .drop("hour", "t"))
res.write.mode("overwrite").parquet(f"{out}/payment_features")
```

Soumettez-le depuis `~/tp3` dans Cloud Shell :

3.A Le calcul (suite)

```
export BUCKET=gs://$(gcloud config get-value project)-shopflow   # si Cloud Shell a redémarré
gcloud dataproc jobs submit pyspark features_spark.py \
  --cluster=shopflow-spark --region=europe-west1 \
  -- gs://shopflow-esigelec-big-data-ia/shopflow_sf10 $BUCKET/tp3/spark
```

Ouvrez l'interface Spark : *Console > Dataproc > Clusters > shopflow-spark > Interfaces web > Spark History Server*. Pour votre job, relevez : le nombre de stages, la taille du *shuffle read/write* de chacun, le *spill* éventuel, la durée de la tâche la plus longue par rapport à la médiane.

3.A Le calcul (suite)

❓ Q4. Dans l'onglet *SQL / DataFrame*, combien d'opérateurs `Exchange` (shuffle) contient le plan ? Associez chacun à une étape du calcul de référence. Lequel déplace le plus de données ? Pour l'agrégation du clickstream, comparez le nombre de lignes en entrée et en sortie du premier `HashAggregate` (agrégation partielle, avant l'`Exchange`) : de combien réduit-il le volume, et pourquoi ?

Sources : 3.A Le calcul

Sources : soumettre un job ; interfaces web ; Spark monitoring ; fonctions de fenêtre Spark SQL.

3.B Un skew à corriger

Script fourni `skew_spark.py` : c'est la première version écrite par un collègue, qui calcule les fenêtres par carte **sur `card_id` brut**, sans la clé `card_key`.

```
import sys
from pyspark.sql import SparkSession, Window, functions as F

spark = SparkSession.builder.getOrCreate()
src, aqe = sys.argv[1], sys.argv[2] == "on"
spark.conf.set("spark.sql.adaptive.enabled", aqe)

pay = (spark.read.parquet(f"{src}/payments")
            .withColumn("t", F.col("payment_ts").cast("long")))
w7 = Window.partitionBy("card_id").orderBy("t").rangeBetween(-7 * 86400, 0)
w30 = Window.partitionBy("card_id").orderBy("t").rangeBetween(-30 * 86400, 0)
result = (pay.withColumn("card_nb_tx_7d", F.count(F.lit(1)).over(w7))
             .withColumn("card_avg_amount_30d", F.avg("amount").over(w30)))
result.agg(F.max("card_nb_tx_7d"), F.avg("card_nb_tx_7d")).show()
```

3.B Un skew à corriger (suite)

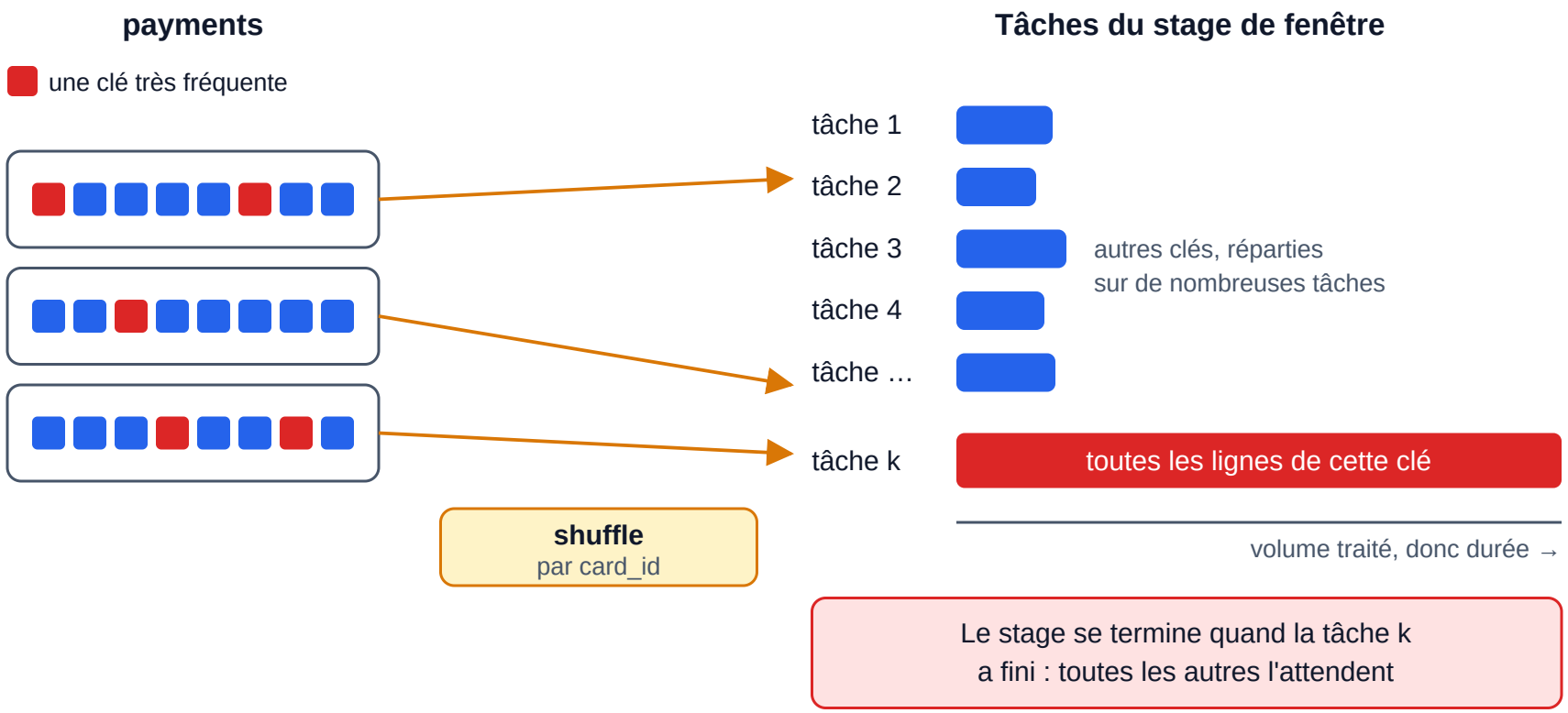


Schéma : la fonction de fenêtre redistribue les paiements par card_id ; toutes les lignes d'une même clé aboutissent dans la même tâche.

3.B Un skew à corriger (suite)

Enregistrez-le dans `~/tp3` de Cloud Shell, comme `features_spark.py`.

1. Lancez-le avec `off` puis avec `on` (dernier argument) sur sf10 :

```
for AQE in off on; do
  gcloud dataproc jobs submit pyspark skew_spark.py \
    --cluster=shopflow-spark --region=europe-west1 \
    -- gs://shopflow-esigelec-big-data-ia/shopflow_sf10 $AQE
done
```

3.B Un skew à corriger (suite)

Comparez les durées, et dans l'interface Spark la distribution des durées de tâches du stage de fenêtre (*Summary Metrics* : min, médiane, max).
2. Regardez le résultat : la valeur maximale de `card_nb_tx_7d` est-elle plausible pour une carte ?
Ce script n'écrit **aucun fichier** : la dernière ligne (`.show()`) affiche un petit tableau, `max(card_nb_tx_7d)` et `avg(card_nb_tx_7d)`, dans la **sortie du driver**. Trois façons de la lire :

- dans le terminal Cloud Shell, à la fin de la commande `gcloud dataproc jobs submit` (le tableau apparaît juste avant `Job [...] finished successfully`) ;
- dans la console : *Dataproc > Tâches* (*Jobs*), cliquez sur le job, onglet *Résultat* (*Output*) ;
- plus tard, depuis Cloud Shell, avec l'identifiant du job :

3.B Un skew à corriger (suite)

```
gcloud dataproc jobs list --cluster=shopflow-spark --region=europe-west1   # repérez le JOB_ID
OUT=$(gcloud dataproc jobs describe JOB_ID --region=europe-west1 --format='value(driverOutputResourceUri)')
gcloud storage cat "$OUT.*" | grep -A4 "max(card_nb_tx_7d)"
```

La sortie du driver est enregistrée dans le bucket de staging de Dataproc, découpée en fichiers `driveroutput.000000000`, `driveroutput.000000001`…

Pour aller plus loin dans l'analyse, ajoutez à la fin du script vos propres agrégations suivies de `.show()` et relancez-le : leur résultat arrive au même endroit.

3. Lancez maintenant la version corrigée, `skew_spark_debug.py`, fournie ci-dessous. Enregistrez-la aussi dans `~/tp3`. Elle ne diffère de `skew_spark.py` que par la clé des fenêtres : `card_key = coalesce(card_id, -payment_id)`, la clé déjà utilisée au TP 2, où chaque paiement sans carte (PayPal) forme sa propre clé.

3.B Un skew à corriger (suite)

```
import sys
from pyspark.sql import SparkSession, Window, functions as F

spark = SparkSession.builder.getOrCreate()
src, aqe = sys.argv[1], sys.argv[2] == "on"
spark.conf.set("spark.sql.adaptive.enabled", aqe)

pay = (spark.read.parquet(f"{src}/payments")
            .withColumn("t", F.col("payment_ts").cast("long"))
            .withColumn("card_key", F.coalesce(F.col("card_id"), -F.col("payment_id"))))   # la correction
w7 = Window.partitionBy("card_key").orderBy("t").rangeBetween(-7 * 86400, 0)
w30 = Window.partitionBy("card_key").orderBy("t").rangeBetween(-30 * 86400, 0)
result = (pay.withColumn("card_nb_tx_7d", F.count(F.lit(1)).over(w7))
             .withColumn("card_avg_amount_30d", F.avg("amount").over(w30)))
result.agg(F.max("card_nb_tx_7d"), F.avg("card_nb_tx_7d")).show()
```

3.B Un skew à corriger (suite)

```
for AQE in off on; do
  gcloud dataproc jobs submit pyspark skew_spark_debug.py \
    --cluster=shopflow-spark --region=europe-west1 \
    -- gs://shopflow-esigelec-big-data-ia/shopflow_sf10 $AQE
done
```

Comparez avec l'étape 1 : durée du job, écart entre la tâche la plus longue et la médiane du stage de fenêtre, et valeur de `max(card_nb_tx_7d)`.

❓ Q5. Quelles lignes forment la partition déséquilibrée ? AQE a-t-il réglé le problème ? Pourquoi (comparez avec un skew dans une jointure, vu en cours) ? En quoi ce skew est-il aussi un **bug** de features, et pas seulement un problème de performance ?

Sources : 3.B Un skew à corriger

Source : Spark SQL performance tuning (AQE, skew join).

Partie 4 : comparaison et décision

Complétez la grille (prix horaires à prendre dans le simulateur de prix GCP pour `europe-west1`) :

Moteur	Machine(s)	SF	Durée	Pic RAM	CPU moyen	Coût estimé (prix/h × durée)	Lignes de code	Facilité de débogage (1-5)
pandas naïf	VM 8 vCPU	1						
pandas naïf	VM 8 vCPU	10						
multiprocessing (8)	VM 8 vCPU	10						
Polars	VM 8 vCPU	10						
Spark	1 + 2 nœuds	10						

❓ Q6. Quel outil recommandez-vous à ShopFlow pour ce recalcul nocturne à sf10 ? Et si le volume était multiplié par 10 ? Reliez votre réponse à la notion de COST vue en cours. Que deviendrait votre choix si les mêmes features devaient être calculées **en continu**, paiement par paiement (séances 4 et 5) ?

Fin de séance : tout supprimer

```
gcloud dataproc clusters delete shopflow-spark --region=europe-west1
gcloud compute instances list          # doit être vide
gcloud dataproc clusters list --region=europe-west1   # doit être vide
```

Gardez le bucket : le cluster Spark sera recréé en séance 4 et vos résultats y restent.

Annexe : monitor.py

```
"""Échantillonne la mémoire utilisée et le CPU de la machine. Usage :
   python monitor.py fichier.csv &        (enregistre jusqu'à kill)
   python monitor.py --resume fichier.csv (affiche le résumé)"""
import csv, sys, time
import psutil

if sys.argv[1] == "--resume":
    rows = list(csv.DictReader(open(sys.argv[2])))
    mem = [float(r["mem_used_gb"]) for r in rows]
    cpu = [float(r["cpu_pct"]) for r in rows]
    duree = float(rows[-1]["t"]) - float(rows[0]["t"])
    print(f"durée {duree:.0f} s | pic RAM {max(mem):.1f} Go (base {mem[0]:.1f}) | CPU moyen {sum(cpu)/len(cpu):.0f} %")
else:
    with open(sys.argv[1], "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["t", "mem_used_gb", "cpu_pct"])
        while True:
            w.writerow([time.time(), psutil.virtual_memory().used / 1e9, psutil.cpu_percent(interval=0.5)])
            fh.flush()
```

Sources : Annexe : `monitor.py`

Source : psutil.

Si quelque chose bloque

Symptôme	Cause probable	Que faire
`Quota 'CPUS' exceeded` ou `Insufficient 'CPUS_ALL_REGIONS' quota`	Trop de vCPU allumés dans le projet (la VM existe encore, ou machines trop grosses)	Supprimer la VM avant de créer le cluster, puis prendre `TYPE=e2-standard-2` (étape 4 de la pause)
Le script pandas est tué sans message (`Killed`)	Mémoire épuisée : le noyau tue le processus (*OOM killer*)	C'est la réponse à Q1 ; vérifier avec `dmesg | tail`
`ValueError` sur `rolling` : index non monotone	Les paiements ne sont pas triés par `payment_ts` dans chaque carte	Trier par (`card_key` , `payment_ts`) avant `rolling`
Résultat multiprocessing différent de la version naïve	Historique de 30 jours ou première heure du mois oubliés	Relire les deux difficultés de la partie 1.B
`Failed to validate permissions required for default service account` à la création du cluster	API Cloud Resource Manager désactivée, ou rôles du compte de service absents	Refaire la fin de l'étape 1 de la partie 0, puis vérifier avec `gcloud dataproc clusters list --region=europe-west1` si le cluster existe déjà
`missing permissions: [storage.objects.get, storage.objects.update] on the staging_bucket`	Le compte de service des VM n'a pas `roles/storage.objectAdmin`	Refaire la fin de l'étape 1 de la partie 0, attendre 2 min, relancer la création
`Permission denied` sur le bucket de l'enseignant	Lecture publique non configurée	Prévenir l'enseignant
Le job Spark ne trouve pas le module `pyspark` en local	Normal : le script s'exécute sur le cluster	Toujours soumettre avec `gcloud dataproc jobs submit`